Advanced Memory

Advanced Programming

Brent van Bladel

Introduction

```
1.  class Circle {
2.    float _radius{1};
3.    Coordinate* _center;
4.   public:
5.    explicit Circle(Coordinate* c) : _center{c} {};
6.    ~Circle() { delete _center; };
7.  };
```

```
1.  Circle c1{new Coordinate(1,3)};
2.  Circle c2{graph.getCoordinate(1,3)};
```

Should the destructor delete _center ?

If the coordinate was created specifically for this circle: yes.

If the coordinate was part of some overarching object: no.

Solution: ownership.

Ownership

A unit of code has **ownership** of a created object if it has the authority on the lifetime of that object.

The owner of an object is responsible for clean-up of the object.

Ownership

```
1.  class Circle {
2.    float _radius{1};
3.    Coordinate* _center;
4.   public:
5.    explicit Circle(Coordinate c) : _center{new Coordinate{c}} {};
6.    ~Circle() { delete _center; }
7.  };
```

Circle has **ownership** of _center.

```
1.  class Circle {
2.    float _radius{1};
3.    Coordinate* _center;
4.   public:
5.    explicit Circle(Coordinate* c) : _center{c} {};
6.    ~Circle() {};
7.  };
```

Unclear who has ownership…
Unclear who has to delete the object…

Solution: smart pointers.

Smart Pointers

A **smart pointer** is pointer-wrapping object that provides additional features, such as memory management.

Smart pointers are implemented in the standard library.

#include<memory>

Smart Pointers: shared_ptr

A **shared pointer** is a smart pointer that retains shared ownership of an object through a pointer.

It automatically counts how many pointers reference the object, and deletes the object when the last reference is destroyed.

Smart Pointers: shared_ptr

```
1.  class Circle {
2.    float _radius{1};
3.    std::shared_ptr<Coordinate> _center;
4.   public:
5.    Circle(std::shared_ptr<Coordinate> c) : _center{c} {};
6.    ~Circle() = default;
7.  };
8.
9.  int main() {
10.   std::shared_ptr<Coordinate> coor
11.                       = std::make_shared<Coordinate>();
12.   Circle c1{coor};
13.   Circle c2{coor};
14. }
```

3 references to the coordinate: coor; c1._center; c2._center

1) **make_shared** creates:

- "coor" on stack
- Coordinate on heap

2) Circle constructor c1 calls the copy constructor of shared_ptr, and stores it (as _center)

3) Circle constructor c2 calls the copy constructor of shared_ptr, and stores it

Smart Pointers: shared_ptr

```
1.  class Circle {
2.    float _radius{1};
3.    std::shared_ptr<Coordinate> _center;
4.   public:
5.    Circle(std::shared_ptr<Coordinate> c) : _center{c} {};
6.    ~Circle() = default;
7.  };
8.
9.  int main() {
10.   std::shared_ptr<Coordinate> coor
11.                       = std::make_shared<Coordinate>();
12.   Circle c1{coor};
13.   Circle c2{coor};
14. }
```

4) c2 is destructed (c2._center is deallocated from the stack)

5) c1 is destructed (c1._center is deallocated from the stack)

6) coor is destructed (deallocated from the stack) → Since this was the last reference to the coordinate on the heap, that coordinate is also deallocated from the heap

Smart Pointers: unique_ptr

An **unique pointer** indicates single exclusive ownership of a resource.

```
1.  class Circle {
2.      std::unique_ptr<Coordinate> _center;
3.    public:
4.      Circle(Coordinate c) : _center{std::make_unique<Coordinate>(c)} {};
5.      ~Circle() = default;
6.  };
```

Smart Pointers: unique_ptr

How is uniqueness enforced?

```
1.  class Circle {
2.      std::unique_ptr<Coordinate> _center;
3.    public:
4.      Circle(std::unique_ptr<Coordinate> c) : _center{c} {};
5.      ~Circle() = default;
6.  };
```

Compilation error: “use of deleted function”!

Deleted functions

A **deleted function** is a function which can not be called.

```
1.  class Circle {
2.      std::unique_ptr<Coordinate> _center;
3.   public:
4.      Circle() = delete;
5.      Circle(Coordinate c) : _center{std::make_unique<Coordinate>(c)} {};
6.      ~Circle() = default;
7.  };
8.
9.  int main(){
10.     Circle c1{}; // Compilation error
11.     Circle c2{Coordinate{1,2}}; // OK
12. }
```

Smart Pointers: unique_ptr

How is uniqueness enforced?
Copy constructor of unique_ptr is a deleted function!

The unique_ptr constructor creates the pointed to object on the heap. You cannot make a copy of the unique_ptr, guaranteeing that only one object has ownership.

What if we want to transfer ownership
by return (giving ownership away) **or**
by parameter (taking ownership) **?**

Returning unique_ptr

What if we want to handle object creation in a method*?

```
1.  std::unique_ptr<Coordinate> createCoordinate(float x, float y) {
2.      return std::make_unique<Coordinate>(Coordinate{x, y});
3.  }
4.
5.  class Circle {
6.      float _radius{1};
7.      std::unique_ptr<Coordinate> _center;
8.   public:
9.      Circle(float x, float y) : _center{createCoordinate(x, y)} {};
10.     ~Circle() = default;
11. };
```

*see Factory Pattern later in this course

Works due to Copy Elision!

Copy Elision

Copy elision is a compiler optimization that prevents extra (potentially expensive) copies in certain situations, resulting in zero-copy pass-by-value semantics.

Since C++17: Copy Elision is guaranteed when an object is returned directly.

Returning unique_ptr

What if we want to handle object creation in a method*?

```
1.  std::unique_ptr<Coordinate> createCoordinate(float x, float y) {
2.      return std::make_unique<Coordinate>(Coordinate{x, y});
3.  }
4.
5.  class Circle {
6.      float _radius{1};
7.      std::unique_ptr<Coordinate> _center;
8.   public:
9.      Circle(float x, float y) : _center{createCoordinate(x, y)} {};
10.     ~Circle() = default;
11. };
```

Constructs the object directly in the return location of the stack frame!

Works due to Copy Elision!

*see Factory Pattern later in this course

Passing a unique_ptr

What if we want to provide a unique_ptr as a parameter to transfer ownership?

```
1.  class Circle {
2.      std::unique_ptr<Coordinate> _center;
3.    public:
4.      Circle(std::unique_ptr<Coordinate> c) : _center{c} {};
5.      ~Circle() = default;
6.  };
```

Compilation error: “use of deleted function”!

Solution: std::move

std::move

The function **std::move** indicates that an object may be "moved from", allowing the efficient transfer of resources.

```
1.  std::string str1 = "Hello";
2.  std::string str2 = "World";
3.  std::vector<std::string> v;
4.
5.  v.push_back(str1);
6.  v.push_back(std::move(str2));
7.  // note that str2 is now empty
```

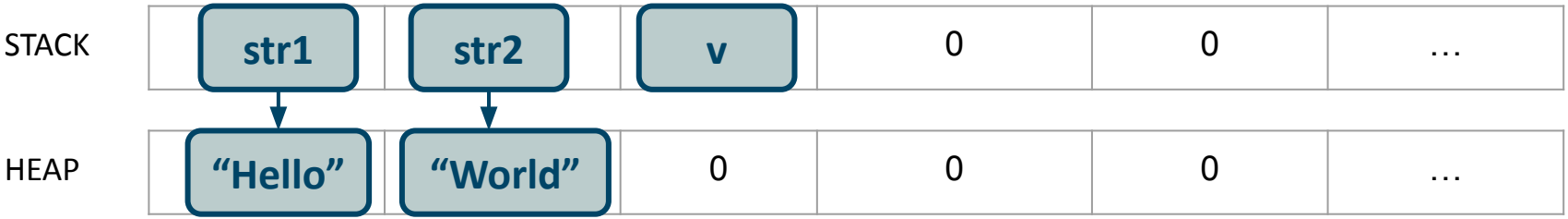
A deep copy of str1 is made and stored in the vector.

A shallow copy of str2 is made, after which str2 is left empty*.

*officially, any std object gets left in an undefined valid state after being moved

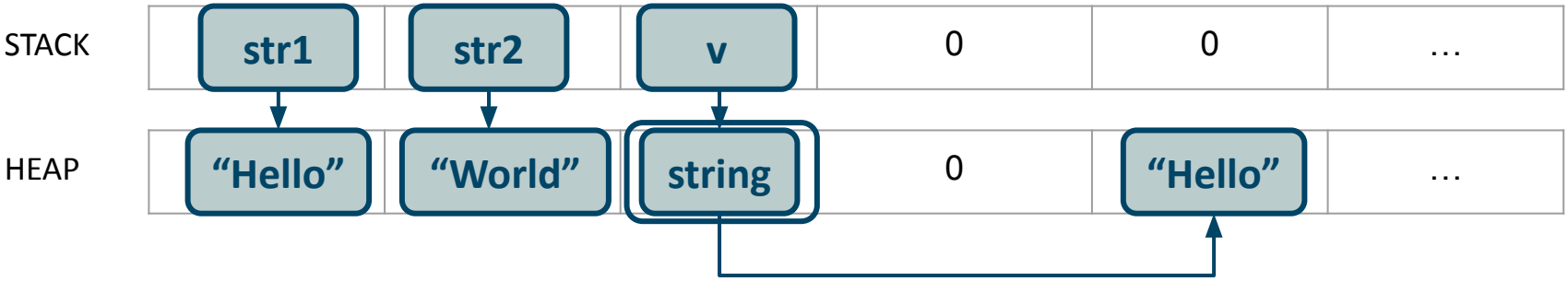
std::move

```
1.  std::string str1 = "Hello";
2.  std::string str2 = "World";
3.  std::vector<std::string> v;
4.
5.  v.push_back(str1);
6.  v.push_back(std::move(str2));
7.  // note that str2 is now empty
```



std::move

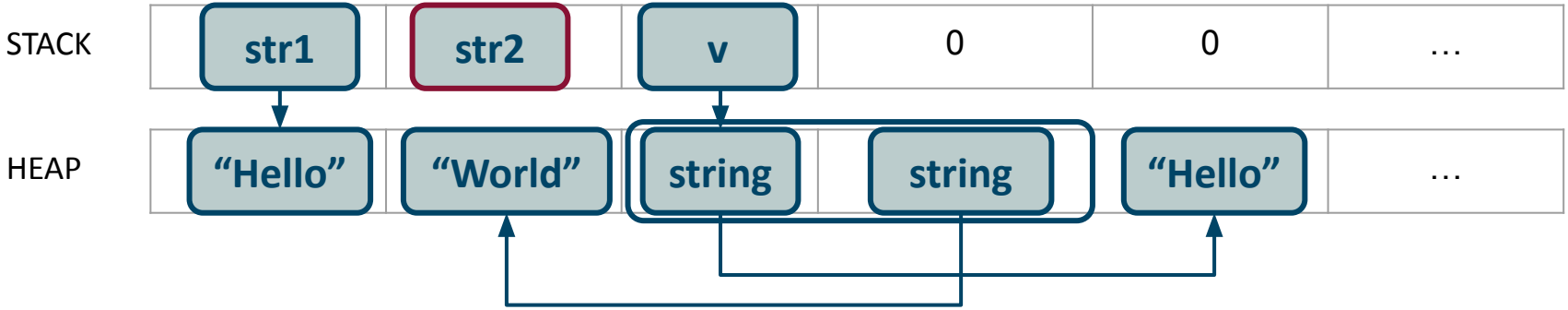
```
1.  std::string str1 = "Hello";
2.  std::string str2 = "World";
3.  std::vector<std::string> v;
4.
5.  v.push_back(str1);
6.  v.push_back(std::move(str2));
7.  // note that str2 is now empty
```



std::move

```
1.  std::string str1 = "Hello";
2.  std::string str2 = "World";
3.  std::vector<std::string> v;
4.
5.  v.push_back(str1);
6.  v.push_back(std::move(str2));
7.  // note that str2 is now empty
```

Note that str2 is not destructed!

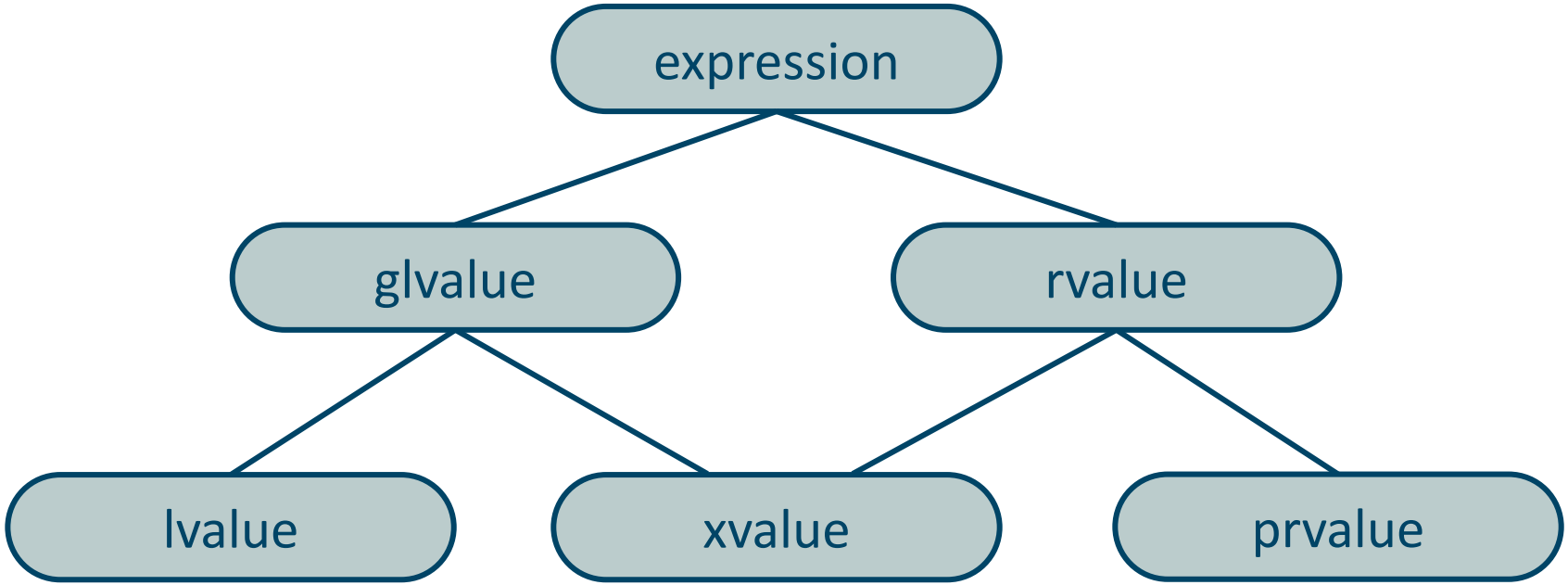


std::move

How does this work?

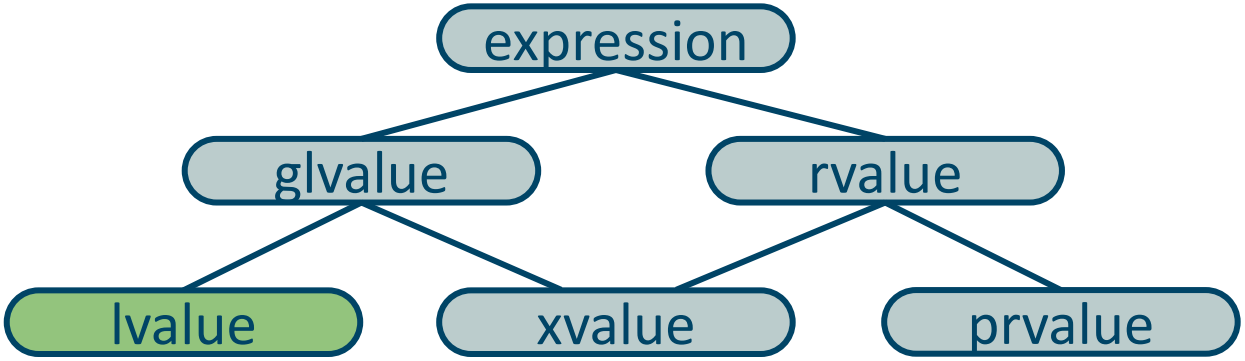
→ **C++ Value Categories**

C++ Value Categories



Every expression is an lvalue, an xvalue, or an prvalue.

lvalue



An **lvalue** is an expression that has an accessible address.

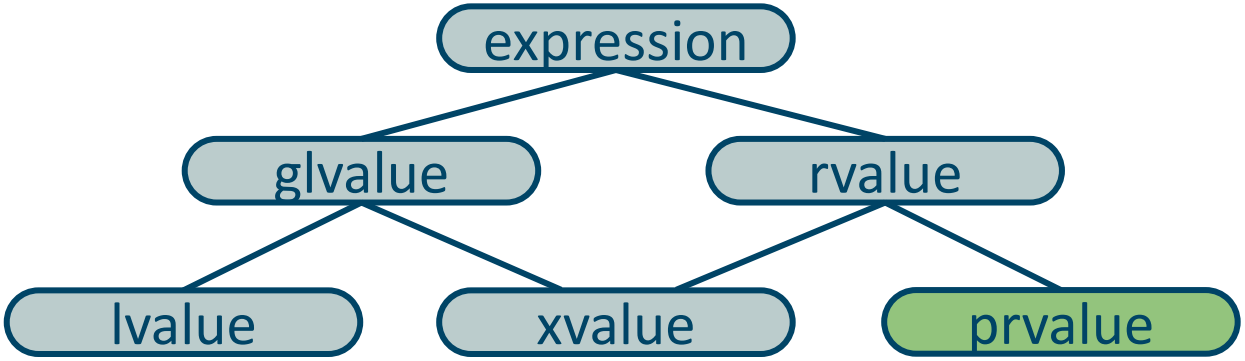
Examples:

- variable names and class members
- functions that return an lvalue reference

```
1.  Coordinate& Circle::getCenter();
```

Note: lvalue stands for “left value”, because the left side of an assignment operation needs to have an address / be an lvalue.

prvalue



A **prvalue** is an expression that has no address.

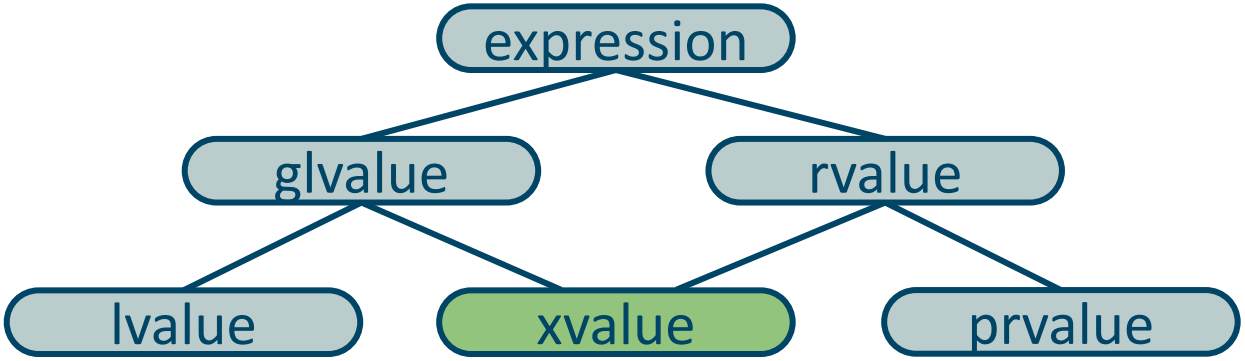
Examples:

- literals
- functions that return a non-reference type

```
1.    float Circle::getRadius();
```

Note: prvalue stands for “pure right value”, because the right side of an assignment operation needs to not have an address for it to be a pure assignment (assignment of value to address).

xvalue



A **xvalue** is an expression that has an inaccessible address.

Example:

- functions that return a rvalue-reference

An rvalue-reference is a reference to an rvalue (xvalue or prvalue).

Note: xvalue stands for “expiring value”, because it usually is an object near the end of its lifetime (so that its resources may be moved).

lvalue reference VS rvalue Reference

References always refer to the object with which they were initialized.

- lvalue references must be initialized with lvalues.

lref and a refer to the same object in memory

- rvalue references must be initialized with rvalues.

rref refers to the temp object in memory

```
1.  int a = 10;
2.
3.  // Declaring lvalue reference
4.  int& lref = a;
5.
6.  // Declaring rvalue reference
7.  int&& rref = circle.getRadius();
```

Referencing a temporary object works due to lifetime extension!

Lifetime Extension of Temporaries

Normally, a temporary object is destroyed at the end of the full expression.

```
1.  int a = 10;
2.  int b = 25;
3.  int c = (a + 10) * (b - 5);
```

However, the lifetime of a temporary object is extended by binding to a reference.

```
1.  int&& d = circle.getRadius();
```

std::move

The function **std::move** changes an lvalue to an xvalue.

```
1.  std::string str1 = "Hello";
2.  std::string str2 = "World";
3.  std::vector<std::string> v;
4.
5.  v.push_back(str1);
6.  v.push_back(std::move(str2));
7.  // note that str2 is now empty
```

Note: the const reference prevents an **additional** copy

```
1.  void push_back (const value_type& val);
2.  void push_back (value_type&& val);
```

Passing a unique_ptr

What if we want to provide a unique_ptr as a parameter to transfer ownership?

```
1.  class Circle {
2.      std::unique_ptr<Coordinate> _center;
3.   public:
4.      Circle(std::unique_ptr<Coordinate>&& c) : _center{std::move(c)} {};
5.      ~Circle() = default;
6.  };
```

Pass it as an rvalue reference and call the move constructor.

Move Constructor

A **move constructor** initializes an object given a different object of the same class by "stealing" that objects resources.

Signature: X(X&&)

```
1.  class Circle {
2.      Coordinate* _center;
3.   public:
4.      // copy constructor
5.      Circle(const Circle& other) : _center{new Coordinate{*other._center}} {};
6.      // move constructor
7.      Circle(Circle&& other) : _center{other._center} {other._center = nullptr; };
8.      // destructor
9.      ~Circle() { if (_center) {delete _center;} }
10. };
```

Move Constructor

```
1.  class Circle {
2.      Coordinate* _center;
3.   public:
4.      // copy constructor
5.      Circle(const Circle& other) : _center{new Coordinate{*other._center}} {};
6.      // move constructor
7.      Circle(Circle&& other) : _center{other._center} {other._center = nullptr; };
8.      // destructor
9.      ~Circle() { if (_center) {delete _center;} }
10. };
```

c1 is passed as a lvalue

c2 is passed as a rvalue

```
1.  Circle c;
2.  Circle c2{c}; // copy constructor
3.  Circle c3{std::move(c2)}; // move constructor
```

Move Assignment

```
1.  class Circle {
2.      Coordinate* _center;
3.   public:
4.      // copy assignment
5.      Circle& operator=(const Circle& other){
6.          _center = new Coordinate{other._center};
7.          return *this;
8.      };
9.      // move assignment
10.     Circle& operator=(Circle&& other){
11.         _center = other._center;
12.         other._center = nullptr;
13.         return *this;
14.     };
15. };
```

A **move assignment** sets the state of an object given a different object of the same class by “stealing” that objects resources.

Signature:
X& operator=(X&&)

Rule of three/five/zero

Rule of three: If a class requires a user-defined destructor, a user-defined copy constructor, or a user-defined copy assignment operator, it almost certainly requires all three.

Rule of five: Any class for which move semantics are desirable, has to declare all five special member functions.

Rule of zero: Other classes should not have custom destructors, copy/move constructors or copy/move assignment operators.

https://en.cppreference.com/w/cpp/language/rule_of_three

Advanced Memory

Quiz

What is the output of this code?

```
1.  int n = 1;
2.  int&& rref = n;
3.
4.  cout << n << endl;
5.  cout << rref << endl;
```

Output: Compilation error:

“cannot bind rvalue reference of type ‘int&&’ to lvalue of type ‘int’”

What is the output of this code?

```
1.  int n = 1;
2.  int&& rref = std::move(n);
3.
4.  cout << n << endl;
5.  cout << rref << endl;
```

Output: “1 1”*

*Although we should be careful using a variable after it was moved!

What is the output of this code?

```
1.  int n = 1;
2.  double&& rref = n;
3.
4.  cout << n << endl;
5.  cout << rref << endl;
```

Output: “1 1”

⟶ rref is bound to an rvalue temporary with value 1.0 (created by implicit conversion)

What is the output of this code?

```
1.  void print(int& x) {
2.      cout << x << endl;
3.  }
4.
5.  int main(){
6.      print(5);
7.  }
```

Output: Compiler error:

"cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'."

Allowing this would be dangerous, since you could then assign a value to a literal.

What is the output of this code?

```
1.  void print(const int& x){
2.      cout << x << endl;
3.  }
4.
5.  int main(){
6.      print(5);
7.  }
```

Output: 5

Lifetime is extended of the temporary literal, because when assigning to a const reference, the compiler is assured the temporary is only read from.

Note: lifetime extension can happen with both lvalue references and rvalue references

What is the output of this code?

```
1.  class Foo {
2.     int x{0};
3.   public:
4.     int getX()    { return x;  };
5.     void print()  { cout << x; };
6.  };
7.
8.  int main(){
9.     Foo f;
10.    int& x_ref = f.getX();
11.    x_ref = 10;
12.    f.print();
13. }
```

Return by value results in a temporary rvalue.

Output: Compiler error:

"cannot bind non-const **lvalue reference of type 'int&'** to an **rvalue of type 'int'**."

What is the output of this code?

```
1.  class Foo {
2.     int x{0};
3.   public:
4.     int& getX()   { return x;  };
5.     void print()  { cout << x; };
6.  };
7.
8.  int main(){
9.     Foo f;
10.    int& x_ref = f.getX();
11.    x_ref = 10;
12.    f.print();
13. }
```

Return by lvalue-reference results in a lvalue.

Output: 10

What is the output of this code?

```
1.  class Foo {
2.     int x{0};
3.   public:
4.     int& getX()   { return x;  };
5.     void print()  { cout << x; };
6.  };
7.
8.  int main(){
9.     Foo f;
10.    f.getX() = 10;
11.    f.print();
12. }
```

Since getX() returns an lvalue, you can assign to it!

Output: 10

Note: can be prevented by returning a const int&

What is the output of this code?

```
1.  int& getNumber() {
2.      int x = 5;
3.      return x;
4.  }
5.
6.  int main(){
7.      cout << getNumber() << endl;
8.  }
```

Output: Undefined behaviour

A **dangling reference** is a reference that outlives the lifetime of the object it refers to.

What is the output of this code?

```
1.  int& getNumber() {
2.      static int x = 5;
3.      return x;
4.  }
5.
6.  int main(){
7.      cout << getNumber() << endl;
8.  }
```

Output: 5

A **static variable** has a static storage duration, and a lifetime from start of program until end of program.

What is the output of this code?

```
1.  int&& getNumber() {
2.      int x = 5;
3.      return std::move(x);
4.  }
5.
6.  int main(){
7.      cout << getNumber() << endl;
8.  }
```

Output: Undefined behaviour.

A rvalue-reference is still a reference, and can be a **dangling reference** as well.

What is the output of this code?

```
1.  int getNumber() {
2.      int x = 5;
3.      return std::move(x);
4.  }
5.
6.  int main(){
7.      cout << getNumber() << endl;
8.  }
```

Output: 5

This function returns a prvalue, which would normally be created by copying the local variable to a temporary. By moving, we allow the temporary to be constructed via the move constructor (shallow copy) instead.

Note: only useful for complex objects

What is the output of this code?

```
1.  int* getNumber() {
2.      int* x = new int(5);
3.      return x;
4.  }
5.
6.  int main(){
7.      cout << *getNumber() << endl;
8.  }
```

Output: 5

Note that this program contains a memory leak!

What is the output of this code?

```
1.  int* getNumber() {
2.      int x = 5;
3.      return &x;
4.  }
5.
6.  int main(){
7.      cout << *getNumber() << endl;
8.  }
```

Output: Undefined behaviour.

Caused by dangling pointer.

What is the output of this code (without SSO)?

```
1.  int main() {
2.      std::string s = "Hello World";
3.      std::string_view sv = s;
4.      std::string s2 = std::move(s);
5.      std::cout << sv;
6.  }
```

Output: “Hello World”.

The characters on the heap are “stolen” by s2, but remain in the same memory location.

What is the output of this code with SSO?

```
1.  int main() {
2.      std::string s = "Hello World";
3.      std::string_view sv = s;
4.      std::string s2 = std::move(s);
5.      std::cout << sv;
6.  }
```

Output: Undefined behaviour.

Characters stored on the stack will need to be **copied**.

→ Std objects will be in a valid but unspecified state after being moved.

What is the output of this code?

```
1.  struct Foo{
2.      Foo(){ std::cout<<"constructor\n"; }
3.      ~Foo(){ std::cout<<"destructor\n"; }
4.      Foo(const Foo & t){ std::cout<<"copy constructor\n"; }
5.      Foo(const Foo&& t){ std::cout<<"move constructor\n"; }
6.  };
7.
8.  int main() {
9.      Foo f;
10.     std::unique_ptr<Foo> p = std::make_unique<Foo>(f);
11. }
```

Output:

constructor

copy constructor

destructor

destructor

What is the output of this code?

```
1.  struct Foo{
2.      Foo(){ std::cout<<"constructor\n"; }
3.      ~Foo(){ std::cout<<"destructor\n"; }
4.      Foo(const Foo & t){ std::cout<<"copy constructor\n"; }
5.      Foo(const Foo&& t){ std::cout<<"move constructor\n"; }
6.  };
7.
8.  int main() {
9.      std::unique_ptr<Foo> p = std::make_unique<Foo>(Foo{});
10. }
```

Output:

constructor

move constructor

destructor

destructor

What is the output of this code?

```
1.  struct Foo{
2.      Foo(){ std::cout<<"constructor\n"; }
3.      ~Foo(){ std::cout<<"destructor\n"; }
4.      Foo(const Foo & t){ std::cout<<"copy constructor\n"; }
5.      Foo(const Foo&& t){ std::cout<<"move constructor\n"; }
6.  };
7.
8.  int main() {
9.      std::unique_ptr<Foo> p = std::make_unique<Foo>();
10. }
```

Output:

constructor

destructor

What is the output of this code?

```
1.  struct Foo{
2.      Foo(){ std::cout<<"constructor\n"; }
3.      ~Foo(){ std::cout<<"destructor\n"; }
4.      Foo(const Foo & t){ std::cout<<"copy constructor\n"; }
5.      Foo(const Foo&& t){ std::cout<<"move constructor\n"; }
6.  };
7.
8.  void print(std::unique_ptr<Foo> ptr);
9.
10. int main() {
11.     std::unique_ptr<Foo> p = std::make_unique<Foo>();
12.     print(p);
13. }
```

Output:

Compilation error: “use of deleted function”

What is the output of this code?

```
1.  struct Foo{
2.      Foo(){ std::cout<<"constructor\n"; }
3.      ~Foo(){ std::cout<<"destructor\n"; }
4.      Foo(const Foo & t){ std::cout<<"copy constructor\n"; }
5.      Foo(const Foo&& t){ std::cout<<"move constructor\n"; }
6.  };
7.
8.  void print(std::unique_ptr<Foo>&& ptr);
9.
10. int main() {
11.     std::unique_ptr<Foo> p = std::make_unique<Foo>();
12.     print(std::move(p));
13. }
```

Output:

constructor

print-output

destructor

What is the output of this code?

```
1.  struct Foo{
2.      Foo(){ std::cout<<"constructor\n"; }
3.      ~Foo(){ std::cout<<"destructor\n"; }
4.      Foo(const Foo & t){ std::cout<<"copy constructor\n"; }
5.      Foo(const Foo&& t){ std::cout<<"move constructor\n"; }
6.  };
7.
8.  int main() {
9.      std::unique_ptr<Foo> p1 = std::make_unique<Foo>();
10.     std::unique_ptr<Foo> p2 = std::move(p1);
11. }
```

Output:

constructor

destructor

Exercise Tip:

Implement your own shared pointer and unique pointer.